

Отчет по практической работе №1

Выполнила: Элякова Алеся Львовна

Группа: БСБО-16-23

Дата выполнения: 08.05.2026

1. Подготовка узлов

1.1 Версия ОС и настройки

```
eal@k8s-master:~$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.4
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
```

1.2 Отключение swap

k8s-master:

```
root@k8s-master:~# free -h
              total        used        free       shared  buff/cache   available
Mem:           3.8Gi        413Mi        3.1Gi         720Ki         504Mi        3.4Gi
Swap:          1.1Gi           0B         1.1Gi
root@k8s-master:~# swapoff -a
root@k8s-master:~# free -h
              total        used        free       shared  buff/cache   available
Mem:           3.8Gi        413Mi        3.1Gi         720Ki         504Mi        3.4Gi
Swap:           0B           0B           0B
root@k8s-master:~#
```

k8s-worker1:

```
root@k8s-worker1:~# free -h
              total        used        free       shared  buff/cache   available
Mem:           1.9Gi        309Mi        1.2Gi         720Ki         511Mi        1.6Gi
Swap:           0B           0B           0B
root@k8s-worker1:~#
```

k8s-worker2:

```
root@k8s-worker2:~# free -h
              total        used        free      shared  buff/cache   available
Mem:          1.9Gi        349Mi        1.2Gi         720Ki         505Mi        1.5Gi
Swap:          0B           0B           0B
```

1.3 Модули ядра

k8s-master:

```
root@k8s-master:~# cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
root@k8s-master:~# modprobe overlay
root@k8s-master:~# modprobe br_netfilter
root@k8s-master:~# lsmod | grep overlay
overlay                217088  0
root@k8s-master:~# lsmod | grep br_netfilter
br_netfilter           36864  0
bridge                 389120  1 br_netfilter
root@k8s-master:~#
```

k8s-worker1:

```
root@k8s-worker1:~# cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
root@k8s-worker1:~# modprobe overlay
root@k8s-worker1:~# modprobe br_netfilter
root@k8s-worker1:~# lsmod | grep overlay
overlay                217088  0
root@k8s-worker1:~# lsmod | grep br_netfilter
br_netfilter           36864  0
bridge                 389120  1 br_netfilter
root@k8s-worker1:~#
```

k8s-worker2:

```
root@k8s-worker2:~# cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
root@k8s-worker2:~# modprobe overlay
root@k8s-worker2:~# modprobe br_netfilter
root@k8s-worker2:~# lsmod | grep overlay
overlay                217088  0
root@k8s-worker2:~# lsmod | grep br_netfilter
br_netfilter           36864  0
bridge                 389120  1 br_netfilter
root@k8s-worker2:~#
```

2. Установка containerd

2.1 Версия containerd

k8s-master:

```
root@k8s-master:~# containerd --version
containerd containerd.io v2.2.3 77c84241c7cbdd9b4eca2591793e3d4f4317c590
root@k8s-master:~#
```

k8s-worker1:

```
root@k8s-worker1:~# containerd --version
containerd containerd.io v2.2.3 77c84241c7cbdd9b4eca2591793e3d4f4317c590
root@k8s-worker1:~#
```

k8s-worker2:

```
root@k8s-worker2:~# containerd --version
containerd containerd.io v2.2.3 77c84241c7cbdd9b4eca2591793e3d4f4317c590
root@k8s-worker2:~#
```

2.2 Конфигурация containerd

k8s-master:

```
eal@k8s-master: ~  
GNU nano 8.4 /etc/containerd/config.toml  
    snapshotter = ''  
    sandboxer = 'podsandbox'  
    io_type = ''  
  
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]  
    BinaryName = ''  
    CriuImagePath = ''  
    CriuWorkPath = ''  
    IoGid = 0  
    IoUid = 0  
    NoNewKeyring = false  
    Root = ''  
    ShimCgroup = ''  
    SystemdCgroup = true  
  
[plugins.'io.containerd.cri.v1.runtime'.cni]  
    bin_dir = ''  
    bin_dirs = ['/opt/cni/bin']  
    conf_dir = '/etc/cni/net.d'
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy

k8s-worker1:

```
eal@k8s-worker1: ~  
GNU nano 8.4 /etc/containerd/config.toml  
    container_annotations = []  
    privileged_without_host_devices = false  
    privileged_without_host_devices_all_devices_allowed = false  
    cgroup_writable = false  
    base_runtime_spec = ''  
    cni_conf_dir = ''  
    cni_max_conf_num = 0  
    snapshotter = ''  
    sandboxer = 'podsandbox'  
    io_type = ''  
  
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]  
    BinaryName = ''  
    CriuImagePath = ''  
    CriuWorkPath = ''  
    IoGid = 0  
    IoUid = 0  
    NoNewKeyring = false  
    Root = ''  
    ShimCgroup = ''  
    SystemdCgroup = true  
  
[plugins.'io.containerd.cri.v1.runtime'.cni]  
    bin_dir = ''  
    bin_dirs = ['/opt/cni/bin']  
    conf_dir = '/etc/cni/net.d'
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy

k8s-worker2:

```

GNU nano 8.4 /etc/containerd/config.toml
privileged_without_host_devices = false
privileged_without_host_devices_all_devices_allowed = false
cgroup_writable = false
base_runtime_spec = ''
cni_conf_dir = ''
cni_max_conf_num = 0
snapshotter = ''
sandboxer = 'podsandbox'
io_type = ''

[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]
  BinaryName = ''
  CriuImagePath = ''
  CriuWorkPath = ''
  IoGid = 0
  IoUid = 0
  NoNewKeyring = false
  Root = ''
  ShimCgroup = ''
  SystemdCgroup = true

[plugins.'io.containerd.cri.v1.runtime'.cni]
  bin_dir = ''
  bin_dirs = ['/opt/cni/bin']
  conf_dir = '/etc/cni/net.d'
  max_conf_num = 1

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line M-E Redo      M-G Copy
Подписать... 5.4 мин 52 Поделиться *** СашаБудык, Василий Еремеев

```

3. Установка kubeadm

3.1 Версии компонентов

k8s-master:

```

root@k8s-master:~# apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@k8s-master:~# kubeadm version
kubelet --version
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"32", GitVersion:"v1.32.13", GitCommit:"6172d7357c6287643350a4fc7e048f24098f2a1b", GitTreeState:"clean", BuildDate:"2026-02-26T20:22:27Z", GoVersion:"go1.24.13", Compiler:"gc", Platform:"linux/amd64"}
Kubernetes v1.32.13
Client Version: v1.32.13
Kustomize Version: v5.5.0
root@k8s-master:~#

```

k8s-worker1:

```

root@k8s-worker1:~# apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@k8s-worker1:~# kubeadm version
kubelet --version
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"32", GitVersion:"v1.32.13", GitCommit:"6172d7357c6287643350a4fc7e048f24098f2a1b", GitTreeState:"clean", BuildDate:"2026-02-26T20:22:27Z", GoVersion:"go1.24.13", Compiler:"gc", Platform:"linux/amd64"}
Kubernetes v1.32.13
Client Version: v1.32.13
Kustomize Version: v5.5.0
root@k8s-worker1:~#

```

k8s-worker2:

```
root@k8s-worker2:~# apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@k8s-worker2:~# kubeadm version
kubelet --version
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"32", GitVersion:"v1.32.13", GitCommit:"6172d7357c6287643350a4fc7e048f24098f2a1b", GitTreeState:"clean", BuildDate:"2026-02-26T20:22:27Z", GoVersion:"go1.24.13", Compiler:"gc", Platform:"linux/amd64"}
Kubernetes v1.32.13
Client Version: v1.32.13
Kustomize Version: v5.5.0
root@k8s-worker2:~#
```

4. Инициализация кластера

4.1 Узлы кластера

```
root@k8s-master:~# kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION
k8s-master	Ready	control-plane	148m	v1.32.13	192.168.248.10	<none>	Debian GNU/Linux 13 (trixie)	6.12.85+deb13-
amd64	containerd://1.7.24							
k8s-worker1	Ready	<none>	102m	v1.32.13	192.168.248.11	<none>	Debian GNU/Linux 13 (trixie)	6.12.85+deb13-
amd64	containerd://1.7.24							
k8s-worker2	Ready	<none>	102m	v1.32.13	192.168.248.12	<none>	Debian GNU/Linux 13 (trixie)	6.12.85+deb13-
amd64	containerd://1.7.24							

```
root@k8s-master:~#
```

4.2 Поды в namespace kube-system

```
^Croot@k8s-master:~# kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-668d6bf9bc-jqb6b	1/1	Running	0	165m
coredns-668d6bf9bc-zpds5	1/1	Running	0	165m
etcd-k8s-master	1/1	Running	0	165m
kube-apiserver-k8s-master	1/1	Running	0	165m
kube-controller-manager-k8s-master	1/1	Running	0	165m
kube-proxy-46xdw	1/1	Running	0	120m
kube-proxy-48psd	1/1	Running	0	120m
kube-proxy-bbmls	1/1	Running	0	165m
kube-scheduler-k8s-master	1/1	Running	0	165m

```
root@k8s-master:~#
```

5. Сетевые компоненты

5.1 Calico

```
eal@k8s-master: ~
```

NAME	READY	STATUS	RESTARTS	AGE
calico-kube-controllers-768fdd9996-vj7x9	1/1	Running	0	117s
calico-node-qpdqw	1/1	Running	0	117s
calico-node-rbjpv	1/1	Running	0	116s
calico-node-shzbs	1/1	Running	0	116s
calico-typha-6c7874d998-2866b	1/1	Running	0	117s
calico-typha-6c7874d998-7z44x	1/1	Running	0	117s
csi-node-driver-82rm9	2/2	Running	0	71s
csi-node-driver-fxsnq	2/2	Running	0	116s
csi-node-driver-mk5kx	2/2	Running	0	62s

5.2 MetalLB

```

root@k8s-master:~# kubectl get pods -n metallb-system
NAME                                READY   STATUS    RESTARTS   AGE
controller-5c8796d8b6-dzvmb        1/1     Running   0           3m53s
speaker-df6vh                      1/1     Running   0           3m53s
speaker-twvlr                      1/1     Running   0           3m53s
speaker-ww8mb                      1/1     Running   0           3m53s
root@k8s-master:~# kubectl get ipaddresspools -n metallb-system
NAME          AUTO ASSIGN  AVOID BUGGY IPS  ADDRESSES
first-pool    true         false            ["192.168.248.100-192.168.248.120"]
root@k8s-master:~#

```

5.3 Ingress Controller

```

root@k8s-master:~# kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-nrgnn 0/1     Completed 0           4m
ingress-nginx-admission-patch-dgkrb   0/1     Completed 0           4m
ingress-nginx-controller-6fb7885497-ls5tc 1/1     Running   0           4m
root@k8s-master:~# kubectl get svc -n ingress-nginx
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
ingress-nginx-controller            NodePort      10.101.68.19  <none>         80:32411/TCP,443:31325/TCP            4m24s
ingress-nginx-controller-admission  ClusterIP     10.110.25.159 <none>         443/TCP                               4m24s
ingress-nginx-controller-lb         LoadBalancer 10.110.182.33 192.168.248.100 80:31288/TCP,443:31483/TCP            2m22s
root@k8s-master:~#

```

6. Развернутое приложение

6.1 Все ресурсы в namespace lab3-app

```

root@k8s-master:~/k8s-lab3# kubectl get all -n lab3-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/go-app-684c68d56d-k4thz        1/1     Running   0           80s
pod/go-app-684c68d56d-rcqjr        1/1     Running   0           84s
pod/go-app-684c68d56d-x82gc        1/1     Running   0           77s
pod/nginx-846d7bc598-kxpgp        1/1     Running   0           7m28s
pod/nginx-846d7bc598-p9cmn        1/1     Running   0           7m28s
pod/postgres-5684b87f55-sn7rp      1/1     Running   0           7m28s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
service/go-app                      ClusterIP      10.101.55.222 <none>         8080/TCP                                7m28s
service/postgres                    ClusterIP      10.107.133.67 <none>         5432/TCP                                7m28s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/go-app              3/3     3             3           7m28s
deployment.apps/nginx               2/2     2             2           7m28s
deployment.apps/postgres            1/1     1             1           7m28s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/go-app-5cf96c8d9b   0         0         0       4m50s
replicaset.apps/go-app-684c68d56d   3         3         3       84s
replicaset.apps/nginx-846d7bc598    2         2         2       7m28s
replicaset.apps/postgres-5684b87f55 1         1         1       7m28s
root@k8s-master:~/k8s-lab3#

```

6.2 PersistentVolumeClaim

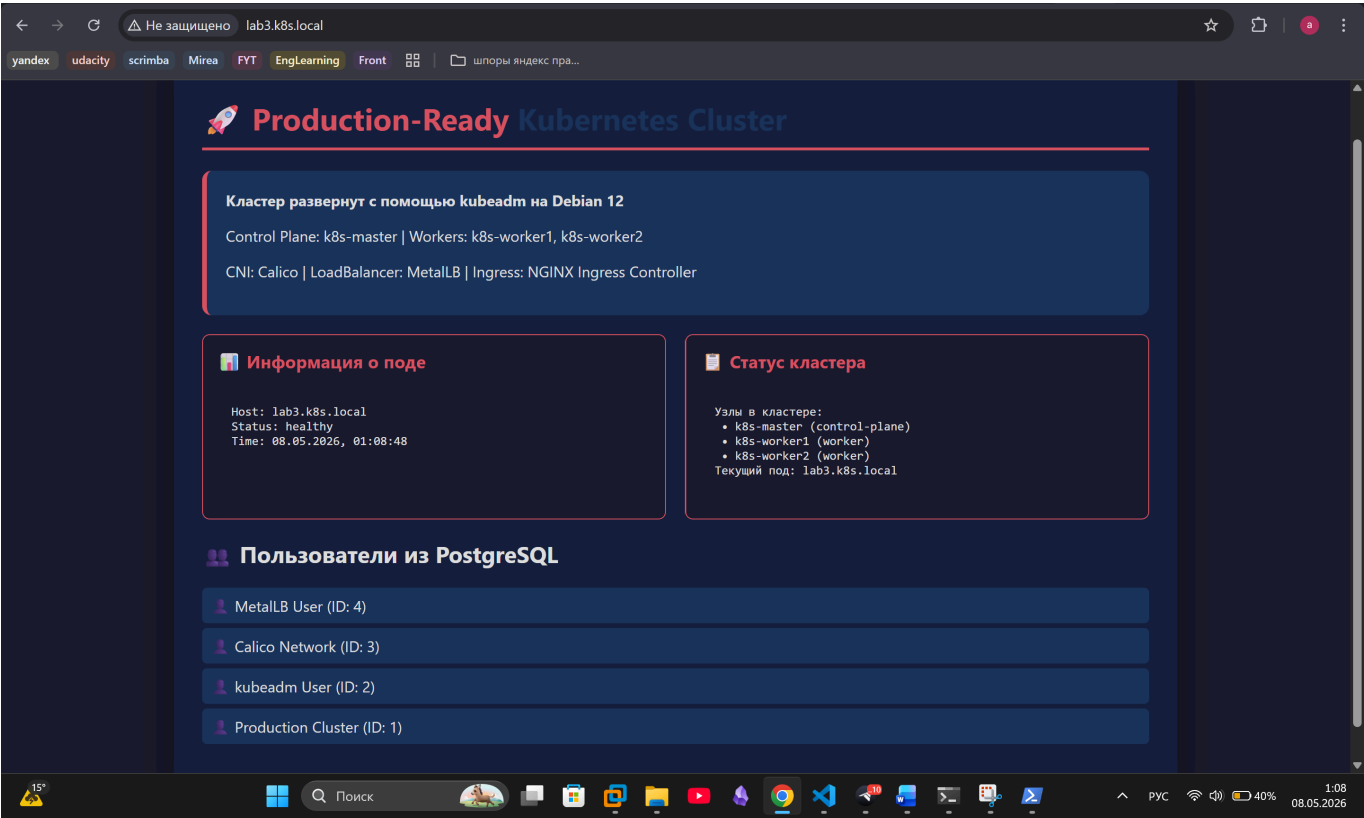
```

root@k8s-master:~/k8s-lab3# kubectl get pvc -n lab3-app
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc  Bound   pvc-dfc6873e-da25-4cf7-bdc5-28b52fbc1045  5Gi        RWO            local-path     <unset>                 8m49s
root@k8s-master:~/k8s-lab3#

```

7. Скриншоты

7.1 Главная страница приложения



7.2 Дашборд с информацией о поде

```
^Croot@k8s-master:~/k8s-lab3# kubectl get pods -n lab3-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
go-app-7c5d466c8f-6qxxg             1/1     Running   0           50m   10.244.126.38   k8s-worker2      <none>            <none>
go-app-7c5d466c8f-7pd69             1/1     Running   0           50m   10.244.194.104 k8s-worker1      <none>            <none>
go-app-7c5d466c8f-f8tfs             1/1     Running   0           50m   10.244.194.103 k8s-worker1      <none>            <none>
nginx-6f8595cf45-qgfh6              1/1     Running   0           2m27s 10.244.194.109 k8s-worker1      <none>            <none>
nginx-6f8595cf45-qj7sm              1/1     Running   0           2m29s 10.244.126.44   k8s-worker2      <none>            <none>
postgres-86dbc5cd7f-d2529           1/1     Running   0           51m   10.244.126.37   k8s-worker2      <none>            <none>
```

7.3 Список пользователей из БД

```
root@k8s-master:~/k8s-lab3# kubectl exec -it -n lab3-app deployment/postgres -- psql -U postgres -d myapp -c "SELECT * FROM users;"
 id |      name      |      created_at
----+-----+-----
  1 | Production Cluster | 2026-05-07 20:37:23.86486
  2 | kubeadm User    | 2026-05-07 20:37:23.86486
  3 | Calico Network  | 2026-05-07 20:37:23.86486
  4 | MetalLB User    | 2026-05-07 20:37:23.86486
(4 rows)
```

8. Тестирование отказоустойчивости

8.1 Симуляция отказа узла


```

root@k8s-master:~/k8s-lab3# kubectl delete pod -n lab3-app -l app=go-app
pod "go-app-7c5d466c8f-6qxxg" deleted
pod "go-app-7c5d466c8f-7pd69" deleted
pod "go-app-7c5d466c8f-f8tfs" deleted
root@k8s-master:~/k8s-lab3# kubectl get pods -n lab3-app -l app=go-app -w
NAME                                READY    STATUS    RESTARTS   AGE
go-app-7c5d466c8f-llrkk             1/1     Running   0           15s
go-app-7c5d466c8f-szkm2             1/1     Running   0           15s
go-app-7c5d466c8f-tqxsx             1/1     Running   0           15s
^Croot@k8s-master:~/k8s-lab3# |

```

8.2 Проверка сохранности данных

```

^Croot@k8s-master:~/k8s-lab3# kubectl delete pod -n lab3-app -l app=postgres
pod "postgres-86dbc5cd7f-d2529" deleted
root@k8s-master:~/k8s-lab3# kubectl apply -f 02-postgres-deployment.yaml
deployment.apps/postgres configured
configmap/postgres-init-sql unchanged
service/postgres unchanged
root@k8s-master:~/k8s-lab3# kubectl exec -it -n lab3-app deployment/postgres -- psql -U postgres -d myapp -c "SELECT * FROM users;"
id |      name      |      created_at
---+-----+-----
 1 | Production Cluster | 2026-05-07 20:37:23.86486
 2 | kubeadm User      | 2026-05-07 20:37:23.86486
 3 | Calico Network    | 2026-05-07 20:37:23.86486
 4 | MetalLB User      | 2026-05-07 20:37:23.86486
(4 rows)

```

9. Ответы на контрольные вопросы

1. Какие компоненты входят в control plane и какова их роль?

- kube-apiserver - центральная точка управления кластером. Все компоненты общаются через него.
- etcd - распределенное хранилище.
- kube-scheduler - распределяет поды по узлам, учитывая ресурсы и ограничения.
- kube-controller-manager - следит за желаемым состоянием кластера и исправляет отклонения (перезапускает упавшие поды и тд)
- cloud-controller-manager - интеграция с облачными провайдерами

2. Чем отличается развертывание с kubeadm от использования Minikube?

	kubeadm	Minikube
Назначение	Реальные кластеры	Локальная разработка
Узлы	Несколько физических/виртуальных машин	Один узел на локальной машине
Настройка	Ручная, гибкая	Автоматическая
Производительность	Полноценная	Ограничена ресурсами ПК
Применение	Продакшн, учебные кластеры	Тестирование, обучение

3. Для чего нужен CNI-плагин и какую роль выполняет Calico?

CNI (Container Network Interface) — стандарт для организации сети между подами. Без CNI поды не могут общаться друг с другом.

Calico конкретно обеспечивает:

- маршрутизацию трафика между подами через BGP
 - сетевые политики (NetworkPolicy) — можно разрешать/запрещать трафик между подами
 - высокую производительность без оверлейных сетей
-

4. В чем разница между Service типа NodePort, LoadBalancer и Ingress?

NodePort — открывает порт на каждом узле кластера (30000-32767). Простой способ, но неудобен в продакшене — нужно знать IP узла и порт

LoadBalancer — запрашивает внешний IP у облака или MetalLB. Каждый сервис получает свой отдельный IP. Удобно, но дорого если сервисов много

Ingress — один внешний IP для всех сервисов, маршрутизация по доменам и путям (`/api` → go-app, `/` → nginx). Самый гибкий вариант для HTTP-трафика

5. Как обеспечивается отказоустойчивость control plane?

В продакшене control plane делают отказоустойчивым через:

- **Несколько мастер-узлов** (обычно 3 или 5) — если один падает, остальные продолжают работу
- **etcd кластер** — данные реплицируются между узлами, кворум обеспечивает консистентность
- **Виртуальный IP** через keepalived/HAProxy — один общий IP для всех мастеров

10. Выводы

В ходе практической работы был развернут Kubernetes кластер с использованием kubeadm на трех виртуальных машинах под управлением Debian 13: один мастер-узел и 2 рабочих узла. В процессе работы возникали некоторые практические проблемы: устаревший GPG ключ репозитория Kubernetes, была исправлена кодировка HTML страницы. Все проблемы были успешно решены.